P1154R1
Type traits for structural comparison

Published Proposal, 2019-01-19

Authors:

Arthur O'Dwyer

Jeff Snyder

Audience:

LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Draft Revision:

5

Current Source:

github.com/Quuxplusone/draft/blob/gh-pages/d1154-comparable-traits.bs

Current:

rawgit.com/Quuxplusone/draft/gh-pages/d1154-comparable-traits.html

---

Abstract

Now that P0732R2 "Class Types in Non-Type Template Parameters" has been adopted, we propose the new type-trait `has_strong_structural_equality<T>`. Code is more robust and maintainable when we can `static_assert` this property.

Table of Contents

§ 1. Strong structural equality should be static-assertable

The concept of "having strong structural equality" that was introduced by [P0732] will become important to programmers. Take this class type for example:

```
template<std::size_t N>
struct fixed_string
{
    constexpr fixed_string(const char (&s)[N+1])
        { std::copy_n(s, N+1, m_data); }
    auto operator<=>(const fixed_string &) const = default;
    char m_data[N+1];
};
```

This type's `operator<=>` is a "structural comparison operator" because it is defaulted, and invokes only other structural comparison operators. Since it yields `strong_ordering`, it also has what we might call "strong structural ordering", although this term is not introduced by P0732. "Strong structural ordering" is a stricter requirement than P0732's "strong structural equality," which is the prerequisite to use a user-defined type as a non-type template parameter.

C++ should permit the programmer to test the presence or absence of this property. Example:

```
static_assert(std::has_strong_structural_equality_v< fixed_string<5> >);
```

This permits maintainability-minded programmers to express their intention in code.

```
template<std::size_t N>
struct broken_fixed_string
{
    constexpr broken_fixed_string(const char (&s)[N+1])
        { std::copy_n(s, N+1, m_data); }
    auto operator<=>(const broken_fixed_string &rhs) const
        { return std::memcmp(m_data, rhs.m_data, N+1) <=> 0; }
    char m_data[N+1];
};
static_assert(std::has_strong_structural_equality_v< broken_fixed_string<5>
    "broken_fixed_string lacks the strong structural equality we expected")

// ... possibly many lines of code here ...
// ... possibly written by a different programmer ...

template<auto V> struct A {};
A<broken_fixed_string("hello")> a;
```

In the snippet above, we get a nice descriptive `static_assert` failure, instead of an unfriendly spew of diagnostics on the line that tries to instantiate `A`.

§ 2. This feature requires support from the compiler

P1154R0 claimed that none of these type-traits could be implemented without a compiler builtin. In fact, `has_strong_structural_equality` *can* be implemented according to standard C++17:

```
template<auto> struct A {};

template<class T, template<T> class = A> using B = void;

template<class T, class = void>
struct HasStrongStructuralEquality : std::false_type {};

template<class T>
struct HasStrongStructuralEquality<T, B<T>> : std::true_type {};

static_assert(HasStrongStructuralEquality< int >::value);
static_assert(!HasStrongStructuralEquality< std::string >::value);
```

This code relies on subtle and maybe-uncertain rules governing when `template<auto> struct A` is a valid argument for a parameter of type `template<T> class`.

- GCC currently does not support this code — that is, they fail the second `static_assert`.
- MSVC flatly rejects it because they don't support `auto` template parameters yet.
- Clang accepts this code. The worst that can be said of Clang is that they give the wrong answer for `HasStrongStructuralEquality<int&&>`, but that can easily be worked around on the library side.

Right now the burden is on the application programmer to know this trivia and come up with workarounds for GCC and MSVC. We propose to simplify the programmer's job by putting this trait into the standard library, where the burden will be on the library to get it right (probably by using a compiler builtin).

§ 3. Provide a full complement of type traits

We propose these six type-traits, with their accompanying `_v` versions. For exposition purposes only, we provide sample implementations in terms of a hypothetical compiler builtin `__has_structural_comparison(T)`.

```
template<class T> struct has_structural_comparison :
    bool_constant< __has_structural_comparison(T) > {};

template<class T> struct has_strong_structural_ordering :
    bool_constant<
        __has_structural_comparison(T) &&
        is_convertible_v<decltype(declval<T>() <=> declval<T>()), strong_ordering>
    > {};

template<class T> struct has_strong_structural_equality :
    bool_constant<
        __has_structural_comparison(T) &&
        is_convertible_v<decltype(declval<T>() <=> declval<T>()), strong_equality>
    > {};

template<class T> struct has_weak_structural_ordering :
    bool_constant<
        __has_structural_comparison(T) &&
        is_convertible_v<decltype(declval<T>() <=> declval<T>()), weak_ordering>
    > {};

template<class T> struct has_weak_structural_equality :
    bool_constant<
        __has_structural_comparison(T) &&
        is_convertible_v<decltype(declval<T>() <=> declval<T>()), weak_equality>
    > {};

template<class T> struct has_partial_structural_ordering :
    bool_constant<
        __has_structural_comparison(T) &&
        is_convertible_v<decltype(declval<T>() <=> declval<T>()), partial_ordering>
    > {};
```

§ 4. Proposed wording

Add six new entries to Table 47 in [meta.unary.prop]:

Template	Condition	Preconditions
`template<class T> struct has_structural_comparison;`	For a glvalue `x` of type `const T`, the expression `x <=> x` either does not invoke a three-way comparison operator or invokes a structural comparison operator (15.9.1).	T shall be a complete type, *cv* `void`, or an array of unknown bound.
`template<class T> struct has_strong_structural_ordering;`	`has_structural_comparison_v<T>` is `true` and the expression `x <=> x` is convertible to `std::strong_ordering`.	T shall be a complete type, *cv* `void`, or an array of unknown bound.
`template<class T> struct has_strong_structural_equality;`	`has_structural_comparison_v<T>` is `true` and the expression `x <=> x` is convertible to `std::strong_equality`.	T shall be a complete type, *cv* `void`, or an array of unknown bound.
`template<class T> struct has_weak_structural_ordering;`	`has_structural_comparison_v<T>` is `true` and the expression `x <=> x` is convertible to `std::weak_ordering`.	T shall be a complete type, *cv* `void`, or an array of unknown bound.
`template<class T> struct has_weak_structural_equality;`	`has_structural_comparison_v<T>` is `true` and the expression `x <=> x` is convertible to `std::weak_equality`.	T shall be a complete type, *cv* `void`, or an array of unknown bound.
`template<class T> struct has_partial_structural_ordering;`	`has_structural_comparison_v<T>` is `true` and the expression `x <=> x` is convertible to `std::partial_ordering`.	T shall be a complete type, *cv* `void`, or an array of unknown bound.

§ 5. LEWG has jurisdiction

Before the San Diego 2018 meeting, the chair of LEWG questioned whether this paper should be handled by SG7 (Reflection and Compile-Time Programming). In response, the chair of SG7 has

drafted D1354R0 "SG7 Guidelines for Review of Proposals," which lists this paper specifically in the "no review required" category. Quoting directly from D1354R0:

> No Review Needed
>
> A type trait that exposes properties of types that are already clearly observable in the behavior of the type within C++ code.
>
> EXAMPLE 6 A trait that exposes strong structural equality.

§ References

§ Informative References

[D1354]

Chandler Carruth. SG7 Guidelines for Review of Proposals. November 2018. URL: http://wiki.edg.com/pub/Wg21sandiego2018/SG7/d1354r0.html

[P0732]

Jeff Snyder; Louis Dionne. Class Types in Non-Type Template Parameters. June 2018. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0732r2.pdf